Partir los datos y describir las películas

Guía para responder a los bloques D (split) y E (features de la torre de películas) del recomendador de CPIA

Arnau Montull para Uriel Martínez

16 de octubre de 2026

Resumen

Esta guía reúne lo necesario para decidir dos cosas: **(D)** cómo partimos los ratings en entrenamiento, validación y test sin hacernos trampas, y **(E)** qué información sobre cada película (sinopsis, géneros, reparto, año…) recibe la torre de ítem. Cada concepto viene con un ejemplo y con cifras medidas en la submuestra de desarrollo (3 154 155 ratings, 20 085 usuarios, 22 787 películas con rating): cuántos usuarios y películas de test no aparecen en entrenamiento con cada tipo de split, cuánta fuga habría, qué cobertura tiene cada feature y qué pasa con las películas de estreno reciente. Al final están las preguntas D1–D10 y E1–E9 con sus opciones, implicaciones (modelo, pérdida, evaluación, entrega E2), qué decisiones del bloque C les afectan y mi recomendación (⋆), que *no* es una decisión. Las cifras salen de `python -m src.analisis.split_y_features` (`reports/split_y_features.md`); la versión corta, con la hoja de respuestas, está en `docs/planning_bloque_D_E.md`.

Índice

1. Dónde estamos y qué se decide aquí **3**
1.1. Lo ya decidido (bloques A, B y C) . . . 3
1.2. La arquitectura que prometimos en E1 . . . 3
1.3. Qué decide cada bloque, y el supuesto que he tomado para E . . . 3

2. Bloque D: cómo partir los datos **4**
2.1. Para qué sirve cada partición . . . 4
2.2. Los cuatro tipos de split . . . 4
2.3. Lo que dicen los datos sobre cada split . . . 5
2.4. El primer día: 59 % de usuarios sin orden real . . . 6
2.5. Fuga silenciosa: lo que se calcula con todos los datos . . . 6
2.6. Usuarios y películas que no están en train . . . 7
2.7. Métricas sobre el catálogo completo y qué K usar . . . 7
2.8. Validación, tamaño de las particiones y semillas . . . 8

3. Bloque E: qué sabe la torre de ítem de cada película **8**
3.1. Para qué sirve la torre de ítem . . . 8
3.2. Cobertura de cada feature (cifras medidas) . . . 8
3.3. Sinopsis: texto a vector . . . 9
3.4. Géneros: la decisión B3 pendiente . . . 9
3.5. Créditos y palabras clave . . . 10
3.6. Año, duración y popularidad . . . 10
3.7. Embedding de `movieId` frente a solo contenido . . . 10
3.8. Cómo se combinan, y normalizar . . . 11
3.9. ¿Congelado o ajustado? . . . 11

4. Las preguntas **11**

Glosario **17**

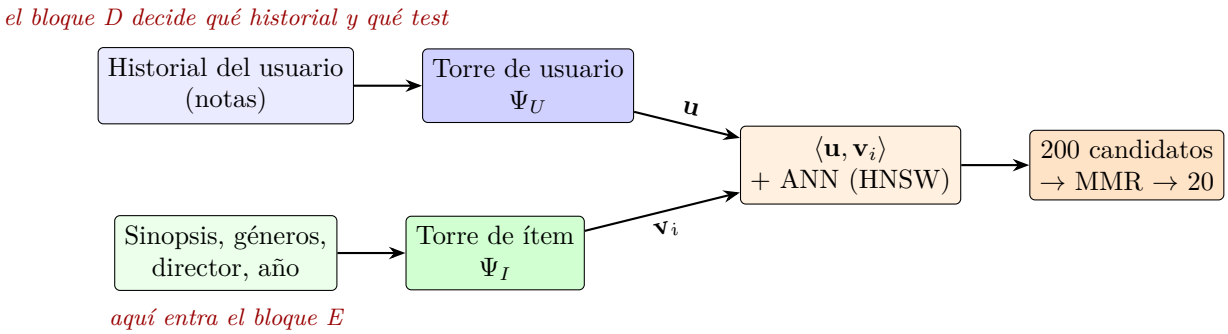
1. Dónde estamos y qué se decide aquí

1.1. Lo ya decidido (bloques A, B y C)

- **A (datos)**: solo `ml-32m/ratings.csv` (1995–2023); de Kaggle/TMDB solo metadatos. Submuestra de desarrollo: el 10 % de los usuarios enteros, estratificada; el conjunto completo, para los resultados finales.
- **B (limpieza)**: fuera las películas con menos de 20 ratings (se pierde el 0,86 % de los ratings); quedan 31 725 154 ratings, 200 948 usuarios y 23 335 películas. Duplicados (mismo `tmdbId`) fusionados. Sinopsis para el 99,5 % de las películas. **B3 sigue pendiente** (géneros de MovieLens o TMDB): lo tratamos en E5.
- **C (señal objetivo)**, provisional salvo lo marcado:

Pregunta	Decisión
C1 b	Implícito binario; pérdida contrastiva con negativos del lote y corrección logQ.
C2	Positivo si $r \geq \min(T; \bar{r}_u)$ con $T = 4{,}0$ **(confirmado)**.
C3 c	Anti-preferencias en la torre de usuario; «nota baja» $= r \leq \bar{r}_u - 1$.
C4 d	Peso $w = 1 + \max(0,\, r - \bar{r}_u)$ multiplicando la pérdida.
C5 a	Cada rating positivo es un ejemplo, sin tope por usuario.
C6 a	$\bar{r}_u$ cruda, calculada solo con train.
C7 c	Relevante en test: con la definición de train y con $r \geq 4{,}0$.
C8 b	Usuarios sin positivos en train o test se quedan en train y salen de las métricas.
C9 c	Peso $\times 0{,}75$ a los ratings de las primeras 24 h de cada usuario.
C10 a, C11 b, C12 a	Nada para la escala; quitar ratings anteriores al estreno; re-ratings: la última.
C13 b	$P(i)$ = positivos de i en train / total, con +1 de suavizado.
C14 c	Enmascarar en la softmax todo lo que el usuario ha valorado en train.

1.2. La arquitectura que prometimos en E1



En E1 prometimos: Two-Tower con pérdida contrastiva y corrección logQ [8], búsqueda aproximada (HNSW) para sacar 200 candidatos, MMR [1] para dejar 20, embedding de la sinopsis con MiniLM ($d = 384$), géneros y director en multi-hot, y el *export* de Letterboxd de un usuario nuevo como caso real.

1.3. Qué decide cada bloque, y el supuesto que he tomado para E

- **Bloque D = split train/validación/test**: cómo se reparten los ratings, qué usuarios y películas ve el modelo al entrenar, con qué se valida, cómo se mide (métricas y K) y cómo se hace reproducible.
- **Bloque E = features de la torre de películas**. Esto no aparece definido en la entrega E1 ni en el PDF de etapas del proyecto de la asignatura; el supuesto sale de `docs/planning_`

`bloques_D_H.md` (bloque E: embedding de sinopsis, créditos, keywords y numéricas). El bloque F (torre de usuario) queda fuera: aquí solo se decide qué sabe el modelo de *cada película*.

Tres cosas que se cruzan

- **D1 y F1** van juntos: un split por usuarios solo tiene sentido si la torre de usuario lee el historial y no un identificador de usuario. Y D1 define el escenario de Letterboxd (un usuario que no estaba en el entrenamiento).
- **D8 y E6** van juntos: el arranque en frío de películas solo se puede *medir* si el split deja películas sin entrenar; y solo se puede *resolver* si la torre de ítem funciona con solo contenido.
- **D3 y E1–E4**: todo lo que se ajusta (vocabularios, frecuencias mínimas, medias para normalizar) tiene que respetar el patrón *fit/transform*.

2. Bloque D: cómo partir los datos

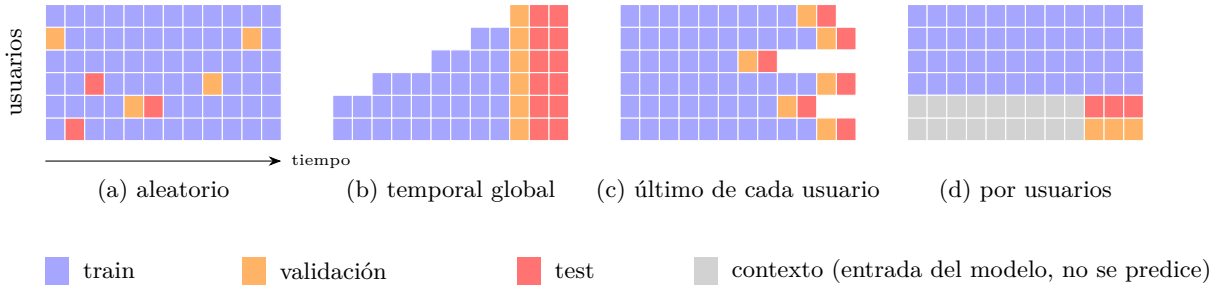
2.1. Para qué sirve cada partición

- **Train**: con él se ajustan los pesos de las torres y todo lo que se «calcula de los datos» ($\bar{r}_u$, $P(i)$ de logQ, la máscara de C14, vocabularios…).
- **Validación**: sirve para elegir hiperparámetros (temperatura, tamaño, tasa de aprendizaje…) y parar el entrenamiento a tiempo (*early stopping*). Cada vez que miramos su resultado y decidimos algo, «se gasta» un poco.
- **Test**: se mira una vez, al final, para decir cuánto vale el modelo. Si se usa para decidir, deja de ser test.

Idea clave

Un split es una *hipótesis sobre el futuro*: dice qué información estará disponible cuando haya que recomendar y qué no. Cuanto más se parezca el split al uso real (un usuario nuevo trae su historial y pide recomendaciones), más honestas son las métricas. En esta guía, ese uso real es el de Letterboxd.

2.2. Los cuatro tipos de split



- **(a) Aleatorio por rating**: cada rating cae al azar en una partición. Fácil, pero entrenamos con ratings posteriores al que predecimos. Es el tipo que [2] descarta porque «filtra información del futuro».
- **(b) Temporal global**: un corte por fecha; lo anterior a train, lo posterior a test. No hay fuga temporal, pero casi todo el test pertenece a usuarios que se registraron después del corte.

- **(c) Último de cada usuario** (*leave-last-out*): su última interacción a test, la anterior a validación. Es el protocolo de NCF y SASRec [4, 5].
- **(d) Por usuarios** (generalización fuerte): un grupo de usuarios entero se aparta del entrenamiento; de cada uno se da al modelo el 80 % de su historial (*fold-in*) y se predice el 20 % restante. Es el protocolo de Mult-VAE y de los trabajos que le siguen [6, 7], y es exactamente el escenario de un usuario nuevo de Letterboxd.

Ejemplo: Ana con cada split

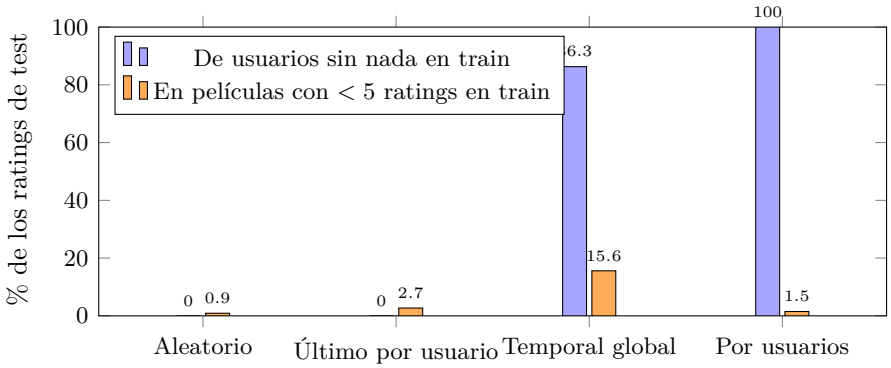
Ana tiene 10 ratings. En (a) el 8 de ellos va a train, uno a val y uno a test *al azar*: su rating más reciente puede estar en train, y uno antiguo en test. En (c) los 8 primeros van a train, el 9.º a val y el 10.º a test. En (b) los que haya antes del corte van a train y los que haya después, a test (si Ana se registró después, *todo* su historial cae en test y el modelo no la ha visto nunca). En (d), o Ana entera es de train, o Ana es de test: los 8 primeros ratings entran en la torre de usuario y el modelo tiene que acertar los 2 últimos.

2.3. Lo que dicen los datos sobre cada split

Todos los splits son 80/10/10 sobre la submuestra de desarrollo, con la semilla 42. «Positivo» es el de C2 ($r \geq \text{mín}(4{,}0; \bar{r}_u)$, con $\bar{r}_u$ de la historia visible).

	Aleatorio	Último/usuario	Temporal global	Por usuarios
Ratings en train	2 523 007	2 505 121	2 523 324	2 528 833
Ratings en val	315 963	324 517	315 429	64 835
Ratings en test	315 185	324 517	315 402	61 867
Usuarios con ratings en test	19 690	20 085	2 259	2 069
Usuarios de test sin nada en train	0	0	1 809	2 069
% ratings de test de usuarios sin train	0	0	86,3	100
% ratings de test en películas sin train	0,06	0,43	13,24	0,15
% ratings de test en películas con < 5 en train	0,90	2,73	15,63	1,54
% ratings de test en películas con < 20 en train	3,32	7,72	19,43	5,35
% de test *anterior* al último rating visible	98,9	0	0	0
% de test el mismo día del último rating visible	1,0	50,2	0	62,2

Corte del temporal global: train hasta el 22/07/2018, validación hasta el 22/10/2020, test después. (En el conjunto completo los cortes son el 01/09/2018 y el 21/10/2020: casi los mismos.)



Lo que enseña:

- **Aleatorio**: el 98,9 % de los ratings de test son *anteriores* al último rating que el modelo ve de ese usuario: se entrena con el futuro. Casi no hay frío, ni de usuarios ni de películas.

- **Temporal global**: el 86,3 % del test son ratings de usuarios que no están en train, y el 13,2 % son de películas sin ningún rating en train. Solo 439 de los 2 259 usuarios de test (19,4 %) tienen algún positivo en test y en su historia: el test mide sobre todo recomendación a usuarios nuevos sin historia. Hay 354 usuarios repartidos entre las tres particiones: son los que llevan años en el sistema.
- **Último por usuario**: realista en el orden, pero el 50,2 % del test se hizo el mismo día que el último rating de train. Y el corte cae en medio de un empate de segundo en el 24,4 % de los usuarios (§2.4).
- **Por usuarios**: el 100 % de los usuarios de test son nuevos para el modelo (como en Letterboxd), solo 0,15 % de los ratings de test son de películas sin entrenamiento, y 2 051 de los 2 069 usuarios de test son elegibles para las métricas (99,1 %).

2.4. El primer día: 59 % de usuarios sin orden real

Recordatorio de la guía C: el 62,5 % de los ratings se hace en las primeras 24 h del usuario, y el 59 % de los usuarios lo valora todo ese primer día. Los autores de MovieLens avisan de que las marcas de tiempo *no* son la fecha de consumo y de que los usuarios introducen muchos ratings de golpe, rellenando lo que vieron antes [3]. Consecuencias para el split:

- «Lo siguiente que ve el usuario» no es lo siguiente que *vio*: es lo siguiente que *puntuó*. Para el 59 % de usuarios, el orden interno es el de teclear la lista de recuerdos.
- En el split «último por usuario» (10 %+10 %) el corte cae dentro de un empate de segundo en el **17,7 %** de los usuarios (train/val) y en el **16,3 %** (val/test): en total, el 24,4 % de los usuarios tiene al menos un corte «arbitrario» (desempata el `movieId`).
- El primer día pesa menos en test que en train (36,7 % de los ratings de test frente al 68,4 % de los de train en el split por último 10 %): se evalúa más con lo posterior, y eso es lo que queremos. Pero con C9 c (peso ×0,75 a las primeras 24 h) el entrenamiento ya *baja* el peso de esos ratings: la evaluación debe ir *sin ponderar*.

¿«Cronológico» es más honesto?

Para los usuarios de varias sesiones, sí. Para el 59 % que lo hace todo de golpe, predecir «el final de la lista de recuerdos» no es predecir el futuro del gusto; es predecir un orden de tecleo. No es una trampa, pero sí es ruido: por eso conviene decir en el informe que el orden dentro del primer día no es información, y repetir el resultado con un split aleatorio dentro del usuario como control (pregunta D2).

2.5. Fuga silenciosa: lo que se calcula con todos los datos

La fuga más peligrosa no es mezclar ratings, sino calcular estadísticas con test sin darse cuenta. En nuestra canalización hay tres: la media del usuario (C6), la frecuencia $P(i)$ de logQ (C13) y la máscara de C14. Con el split «último por usuario» medimos qué pasaría si se calcularan con todo:

- $\bar{r}_u$ con todo el historial frente a solo train: diferencia absoluta media de 0,062 (p95 0,178, máx. 0,92).
- El **1,46 %** de los ratings de train (36 534) cambiaría de etiqueta positivo/no positivo, y el 1,25 % de los de test.
- 1 730 películas tienen algún positivo en la muestra completa pero ninguno en train. Con $P(i)$ calculada con todo se les daría un $\log P(i)$ finito que solo existe porque hay positivos en test

(la correlación de Spearman entre ambos conteos es 0,972: no es un desastre, es un sesgo pequeño pero injustificable).

- La máscara de C14 (lo valorado *en train*) no puede incluir lo valorado en test: sería quitar de la softmax justo las películas que hay que acertar.

La regla: ajustar con train y transformar el resto (patrón *fit/transform*). Vale también para vocabularios (directores, actores), la frecuencia mínima de E2 y las medias y desviaciones con las que se normalizan año y duración (E4).

2.6. Usuarios y películas que no están en train

Usuarios: en el split por usuarios, todos los de test son nuevos por construcción; el modelo solo los conoce por su historial (contexto). Eso exige una torre de usuario que lea el historial (bloque F). En el temporal global, el 86,3 % del test es de usuarios sin historia visible, y no se les puede evaluar con ninguna métrica de recomendación personalizada.

Películas: en nuestros splits con usuarios en train, apenas el 0,15 %–0,43 % de los ratings de test son de películas que train no ha visto nunca (13,2 % en el temporal global). Con eso, *no podemos medir el arranque en frío de películas* (la promesa de E1) con un split de ratings: tendríamos que retirar películas adrede (pregunta D8). Y ojo con la submuestra: el filtro de 20 ratings se aplicó al conjunto completo; en la submuestra, 15 422 películas de las 23 335 tienen menos de 20 ratings en train, y 4 354 no tienen ningún positivo ($P(i)$ solo existe por el suavizado +1). En el conjunto completo, con diez veces más usuarios, habrá menos películas en esa situación: el frío de la submuestra es peor que el real.

2.7. Métricas sobre el catálogo completo y qué K usar

Las métricas son las de la guía C: Recall@K y NDCG@K [6] (esta última, premiando acertar arriba), sobre *todo* el catálogo (22 787 películas en la submuestra, quitando lo que el usuario ya tiene en su historial visible), no sobre 100 negativos al azar. Esa práctica de muestrear 100 negativos es la que usan SASRec y NCF por coste [5, 4], pero aquí el catálogo cabe: 2 056 usuarios de validación $\times$ 22 787 películas son 47 millones de productos escalares. Además de las dos de ranking, se miden **cobertura@K** (qué fracción del catálogo llega a aparecer en alguna lista) y **novedad@K** (por ejemplo, $-\log_2 P_{\text{train}}(i)$ media de lo recomendado).

¿Qué K? El sistema tiene dos etapas: HNSW devuelve 200 candidatos y MMR deja 20. Cada etapa tiene su K: Recall@200 mide si la primera etapa trae los relevantes (es lo que entrena la Two-Tower); NDCG@20 y Recall@20 miden la lista final (y se calculan *antes y después* del MMR). Un K demasiado pequeño choca con cuántos relevantes hay: el **techo** de Recall@K es la media de $\text{mín}(K, |R_u|)/|R_u|$ (el máximo que sacaría un modelo perfecto).

Techo de Recall@K (%)	Aleatorio	Último/usuario	Temporal global	Por usuarios
$K = 10$	87,5	88,7	55,1	77,1
$K = 20$	95,4	96,0	71,1	89,9
$K = 50$	99,3	99,4	86,9	97,8
$K = 200$	100	100	98,5	100
Positivos por usuario elegible (mediana)	5	5	20	9
% de elegibles con < 3 positivos en test	24,9	20,0	10,3	6,5
Usuarios elegibles	18 759	19 396	439	2 051

En el split por usuarios, un Recall@10 perfecto es 77,1 %: si se compara Recall@10 con la literatura, hay que tener presente este techo. Con 2 051 usuarios elegibles y 35 019 positivos en test, el ruido

por usuario es alto; hay que acompañar cada métrica con un intervalo (*bootstrap* sobre usuarios).

2.8. Validación, tamaño de las particiones y semillas

- **Validación**: lo más coherente es una validación *del mismo tipo que el test*: usuarios apartados (2 056 en la submuestra) con su 80 % de historia y el 20 % a predecir, para parar el entrenamiento y elegir hiperparámetros. Así validamos lo que vamos a usar (usuario nuevo con historial).
- **Tamaño**: Mult-VAE aparta 10 000 usuarios para validación y otros 10 000 para test de 136 677 (el 7,3 % cada uno en ML-20M) [6]. Con 80/10/10 nuestro conjunto completo da ≈20 000 usuarios de validación y otros tantos de test.
- **Semillas**: la semilla cambia qué usuarios caen en test. Con 5 semillas, en el split por usuarios los ratings de test varían $66\,565 \pm 1\,569$, los usuarios elegibles $2\,005 \pm 38$ y el % de test en películas con < 5 ratings en train $1{,}8 \pm 0{,}5$; en el aleatorio las cifras apenas se mueven ($315\,203 \pm 223$). Con tan pocos usuarios de test, un resultado con una sola semilla se parece a una moneda; repetir con varias semillas es parte del método.
- **Submuestra o completo**: la submuestra es útil para iterar; el conjunto completo da más usuarios de test (menos ruido) y menos frío espurio. Los cortes temporales globales de ambos coinciden casi (julio–sept. de 2018 y octubre de 2020).

3. Bloque E: qué sabe la torre de ítem de cada película

3.1. Para qué sirve la torre de ítem

La torre de ítem Ψ_I convierte cada película en un vector $\mathbf{v}_i \in \mathbb{R}^{128}$. Los $\mathbf{v}_i$ no dependen del usuario: se calculan una vez para todo el catálogo y se guardan en el índice HNSW. El contenido es lo que permite que una película *sin ratings* tenga un vector razonable: la razón para usar la arquitectura en el catálogo con cola larga [8]. Yi et al. describen una torre de ítem que codifica «una gran variedad de características de contenido» y la proponen precisamente para la escasez de datos y la distribución de potencias de los ítems.

3.2. Cobertura de cada feature (cifras medidas)

Feature	% películas	% ratings
Géneros de MovieLens	99,0	99,9
Géneros de TMDB	98,7	99,8
Géneros de alguna de las dos	99,8	100,0
Sinopsis	99,5	99,9
Director (`credits.csv`)	84,8	97,0
Reparto (`credits.csv`)	83,7	96,9
Palabras clave (`keywords.csv` o API)	84,7	98,5
Año de estreno	99,7	99,9
Duración (`runtime` de Kaggle)	84,7	—

« % ratings» pondera cada película por sus ratings en la submuestra: las que más pesan en el entrenamiento están casi todas completas.

Las películas nuevas son justo las que no tienen créditos

Estreno	Películas	% director	% reparto	% keywords	% sinopsis
≤1989	6 336	98,0	97,1	87,2	99,7
1990–99	3 424	97,5	96,5	81,8	99,3
2000–09	5 432	97,8	96,3	82,5	99,5
2010–16	4 709	97,7	96,2	80,8	99,5
≥2017	3 361	**9,2**	**9,0**	91,9	99,2

`credits.csv` es de Kaggle (2017): las películas posteriores tienen sinopsis y géneros (los pone la API de TMDB que descargamos) pero casi nunca director ni reparto, ni duración (8,9 %). Si director y reparto entran como feature, el modelo ve «desconocido» exactamente en las películas más recientes, que son las del arranque en frío. Hay que decidir qué hacer con ello (E9).

3.3. Sinopsis: texto a vector

La sinopsis tiene una mediana de 46 palabras (p5: 16; p95: 114; el 0,5 % tiene menos de 10). Cabe sobradamente en la ventana de MiniLM (`all-MiniLM-L6-v2`, 256 *word pieces*, ≈190 palabras). Fuente: 19 804 de Kaggle, 3 391 de la API, 13 corregidas a mano y 127 sin fuente (sinopsis vacía).

- **MiniLM**: red preentrenada que da un vector de 384 números por texto; se ejecuta una vez y se guarda (`.npy`). 23 335 × 384 son 17,9 MB en `float16` y 35,8 MB en `float32`.
- **TF-IDF + SVD**: cuenta palabras ponderadas por su rareza y las comprime con una descomposición en valores singulares; no usa red neuronal y es rapidísimo, pero no sabe que «car» y «automobile» son lo mismo.
- Estas representaciones son *del texto*, no de los ratings: ajustar el idf o la SVD con el catálogo entero no filtra información de test (sí lo haría un vocabulario construido con los positivos). Para el experimento de películas retiradas (D8), conviene ajustarlas sin esas películas, por pulcritud.

Sobre el modelo MiniLM no cito un artículo: no está en `docs/papers` y no lo he podido comprobar aquí; en la E2 habrá que referenciar el modelo de la biblioteca *sentence-transformers* con su ficha oficial.

3.4. Géneros: la decisión B3 pendiente

	MovieLens	TMDB
Géneros distintos en el catálogo	19	23
Géneros por película (media)	2,09	2,36
Películas con géneros	99,0 %	98,7 %
Películas con géneros solo en esta fuente	258	201

Sobre los 16 géneros comunes (Sci-Fi = Science Fiction, Children ≈ Family), en las 22 838 películas que tienen las dos listas el Jaccard medio es **0,75** y las listas son idénticas solo en el **50,2 %**. Hay 38 películas sin géneros en ninguna. Solo MovieLens: Musical, Film-Noir, IMAX (un formato de proyección) y Children. Solo TMDB: Family, Music, History, Foreign (indica que no es estadounidense) y TV Movie.

El argumento decisivo no es la cobertura: es la **inferencia**. Las películas nuevas (posteriores a 2023, y las de Letterboxd que no están en MovieLens) no tienen géneros de MovieLens, pero sí los

tienen en TMDB. Si la torre se entrena con géneros de MovieLens, hay que reemplazarlos por otros al servir.

3.5. Créditos y palabras clave

- **Director**: 7 827 directores distintos; el 40,5 % de las películas tiene un director con 5 películas o más en el catálogo.
- **Reparto (3 primeros)**: 22 100 actores distintos; solo 1 114 salen en 10 películas o más, y cubren el 36,4 % de las apariciones. Si se hace multi-hot de todos, la mayoría de las columnas aparecen una vez: el vector no generaliza. Dos soluciones: frecuencia mínima (los demás van a `<OTRO>`) o *feature hashing* a un tamaño fijo.
- **Palabras clave**: aportan matiz temático («heist», «time travel») pero se solapan con la sinopsis; se pueden añadir al texto antes de pasarlo por MiniLM (sin columnas nuevas).

3.6. Año, duración y popularidad

- **Año y duración**: baratos y con buena cobertura (99,7 %; 84,7 %, con mediana de 100 min y p5–p95 de 70–142). Hay que normalizarlos con la media y desviación de train (o de catálogo: son metadatos), o agruparlos por décadas.
- **Popularidad** (nº de ratings, `vote_count`...): el 10 % de películas más vistas reúne el 78,8 % de los ratings y allí la tasa de positivos es más alta (53,3 % de $r \geq 4$ en el primer decil frente al 30–40 % del resto). La popularidad ya entra en el entrenamiento por las frecuencias de las películas positivas; con logQ, el modelo estima la verdadera $p(i \mid u)$ incluyendo la popularidad. Añadirla como feature da al modelo un atajo para acertar recomendando siempre lo mismo, justo el problema que E1 dice combatir.
- Una analogía: YouTube mete la «edad del ejemplo» como feature al entrenar y la pone a cero al servir, para que el modelo no aprenda un sesgo del pasado [2]. La popularidad podría tratarse igual (un sesgo b_i solo en entrenamiento); no es lo mismo que logQ, pero comparte la idea.
- Correlación de Spearman entre nº de ratings y año: $-0{,}14$. Las películas de los años 90 tienen la mediana más alta (370 ratings); las de los 2010 y 2020, 70 y 56.

3.7. Embedding de `movieId` frente a solo contenido

Un embedding por película (tabla $23\,335 \times 128$, casi 3 millones de parámetros) deja que el modelo memorice qué gusta a quién. Pero con 1 491 641 positivos de train en la submuestra hay $\approx 0{,}50$ positivos por parámetro, y 12 872 películas tienen menos de 5 positivos (4 354, ninguno): su embedding casi no se entrena y su representación sería ruido. La torre de ítem de YouTube combina identificador de vídeo y características de contenido [8]. Alternativas:

- **Solo contenido**: funciona con películas nuevas, pero pierde la señal colaborativa fina.
- **Contenido + ID**: lo mejor para las populares, pero para una película sin ID entrenado el vector depende de un embedding aleatorio.
- **Contenido + ID con *dropout* del ID**: se apaga el ID al azar durante el entrenamiento para que el contenido tenga que sostener el vector; al servir, una película nueva usa un ID «desconocido».

No hay datos que decidan esto sin entrenar: es un experimento (con la prueba de D8).

3.8. Cómo se combinan, y normalizar

El vector de ítem sale de una red densa sobre la concatenación de [sinopsis (384), géneros (multi-hot), embeddings de director y actores, año, duración, ¿ID?]. Yi et al. encuentran que **normalizar con L2** los vectores de usuario y de ítem y añadir una temperatura mejora la entrenabilidad y la calidad [8]. Para nosotros hay una razón extra: el MMR usa la similitud $\mathrm{Sim}(\mathbf{v}_i, \mathbf{v}_j)$ (coseno) y HNSW busca por producto escalar: con vectores normalizados las dos cosas coinciden.

3.9. ¿Congelado o ajustado?

MiniLM puede quedarse congelado (se precalcula, se cachea y no entra en el bucle de entrenamiento) o ajustarse (*fine-tuning*). Ajustarlo obliga a recalcular los embeddings cada vez que se actualiza el modelo y complica leer los `.npy` desde C++ en inferencia. Un compromiso: congelado y una capa densa entrenable por encima.

4. Las preguntas

$\star$ = mi recomendación, no una decisión. Las preguntas D1–D3 y E1–E4 vienen de `docs/planning_bloques_D_H.md` (15/10/2026), con su numeración y su texto; las demás las añado yo. «Afecta C» = decisiones de C relacionadas.

Bloque D: split

D1. Tipo de split

F1, D4, D8

¿Cómo partimos?

a) Aleatorio por rating. El 98,9 % del test es anterior al último rating visible: infla las métricas.

b) Temporal global (test = después del 22/10/2020 aprox.). El 86,3 % del test es de usuarios sin historia; solo 439 usuarios elegibles.

c) Último de cada usuario (*leave-last-out*). Estándar en NCF/SASRec. En el 24,4 % de usuarios el corte cae en un empate.

d) $\star$ Por usuarios: 80/10/10 de usuarios, 80 % de su historia como contexto y 20 % a predecir. Es el escenario Letterboxd.

Modelo: (d) exige que la torre de usuario lea el historial (F1 b, no un `userId`). **Pérdida**: solo los ratings de usuarios de train generan ejemplos; el contexto de val/test no. **Evaluación**: solo en (d) todos los usuarios de test son nuevos y hay 2 051 elegibles. **E2**: etapa `03_split` y tabla `usuario-grupo` como artefacto.

Afecta C: C1 b (los positivos de test dependen del split), C5 a (los ejemplos son solo de los usuarios de train), C8 b, C14 c (la máscara se construye con el historial visible).

D2. Orden dentro del usuario

D5, C9

En los usuarios de test, ¿el 80/20 es cronológico o aleatorio?

a) $\star$ Cronológico (primer 80 % como contexto, último 20 % a predecir), con desempate fijo por `movieId`.

b) Aleatorio dentro de cada usuario.

c) Cronológico y, como control, repetir con (b).

Evaluación: en el split por usuarios, el 62,2 % de los ratings a predecir se hizo el mismo día que el último de contexto; el 62,5 % de todos los ratings es del primer día. **Pérdida**: ninguna. **E2**: parámetro `-orden {cronologico,aleatorio}`. (c) cuesta una segunda pasada; solo se justifica si la diferencia es grande.

Afecta C: C9 c (el peso 0,75 al primer día es de entrenamiento; la evaluación va sin pesos), C12 a.

D3. Estadísticas solo con train

C6, C13, E2, E4

¿Qué se calcula solo con los datos de entrenamiento?

a) ⋆ Todo lo «aprendido de los datos» se ajusta con train (patrón *fit/transform*); para el modelo final, se vuelve a ajustar con train+val.

b) Siempre solo con train, también en el modelo final.

c) Con todo: no.

Qué entra: $\bar{r}_u$ (C6), $P(i)$ (C13), máscara (C14), vocabulario y frecuencia mínima de directores/actores (E2), media y desviación de año y duración (E4). **Fuga medida** (si se hiciera con todo): 1,46 % de etiquetas de train cambian y 1 730 películas reciben un $P(i)$ gracias a test. **E2**: cada transformador guarda sus parámetros ajustados en un artefacto.

Afecta C: C6 a, C13 b, C14 c.

D4. Validación

D1, D7

¿Con qué se eligen hiperparámetros y se para el entrenamiento?

a) ⋆ Usuarios de validación apartados (como el test: 80 % contexto, 20 % a predecir), 10 % de los usuarios.

b) Un 10 % de los ratings de train, al azar.

c) Validación cruzada por usuarios (k carpetas).

d) Validar solo con una submuestra de los usuarios de validación (más rápido).

Evaluación: criterio de parada, Recall@200 o NDCG@20 sobre el catálogo completo (47 millones de productos escalares por pasada en la submuestra, asumible). (b) mezcla el futuro en el entrenamiento y no valida «usuario nuevo». (c) multiplica el coste por k.

Afecta C: C7 c (la validación también tiene las dos definiciones de relevante), C8 b.

D5. Primer día y empates

C9, D2

¿Qué hacemos con el 59 % de usuarios que lo valoran todo el primer día?

a) ⋆ Nada especial en el split: aceptar el orden y desempatar con `movieId`; documentar que el orden dentro del primer día no es información.

b) Que el objetivo sean solo ratings posteriores a las primeras 24 h (solo usuarios con varias sesiones).

c) Para los usuarios de «un solo día», 80/20 aleatorio; para el resto, cronológico.

d) Quitar a los usuarios cuyo corte cae en un empate de segundo (24,4 %).

Evaluación: (b) deja un test que mide otra cosa (ya no es Letterboxd) y menos usuarios; (d) tira casi un cuarto de los usuarios. (c) es defendible pero añade una regla que explicar. **Pérdida**: ya se trata con C9 c.

Afecta C: C9 c, C12 a.

D6. Métricas y K C7, F, MMR

¿Qué se mide y con qué K?

a) Recall@20 y NDCG@20.

b) Recall@200 y NDCG@20.

c) ⋆ $K \in \{10, 20, 50, 200\}$ más cobertura@20 y novedad@20, siempre sobre el catálogo completo; principal: Recall@200 (retrieval) y NDCG@20 (lista final, antes y después del MMR).

d) Métricas con 100 negativos muestreados.

Evaluación: techo de Recall@10 en el split por usuarios: 77,1 %; @20: 89,9 %; @200: 100 %. (d) no es comparable entre modelos (guía C). **Pérdida**: ninguna, pero las métricas se calculan con las dos definiciones de C7 c. **E2**: intervalos de confianza por *bootstrap* de usuarios.

Afecta C: C7 c, C8 b (los usuarios sin positivos salen), C14 c (en inferencia se excluye lo ya valorado).

D7. Tamaño de las particiones D1, D9

¿Qué fracción de usuarios para val y test?

a) ⋆ 80/10/10 de usuarios (en la submuestra, ≈2 000 usuarios en val y en test; en el completo, ≈20 000).

b) 90/5/5.

c) Un número fijo de usuarios (p. ej. 10 000 como en Mult-VAE [6]) en el conjunto completo.

Evaluación: con 2 051 elegibles y 35 019 positivos hay ruido (la semilla mueve ±38 usuarios elegibles); más usuarios de test = menos ruido, a costa de menos entrenamiento. **Modelo**: con (a) hay 15 960 usuarios de entrenamiento en la submuestra.

Afecta C: C5 a, C8 b.

D8. Arranque en frío E6, D1

¿Qué hacemos con películas y usuarios de test sin historia en train?

a) Ignorarlo (dejarlos en el test tal cual).

b) Quitar del test las películas sin train (solo el 0,15 % en el split por usuarios).

c) Reportar aparte: métricas globales y métricas sobre ratings en películas con < 5 (o < 20) en train.

d) ⋆ c y, además, un experimento de películas retiradas: quitar todos los ratings de train de un 5 % de películas y medir sobre ellas.

Evaluación: con el split por usuarios solo el 1,54 % del test es de películas con < 5 ratings en train: sin el experimento d, la promesa de arranque en frío de E1 no se puede probar. **Usuarios**: Letterboxd ya es el escenario del split (d) de D1. **Modelo**: la prueba decide E6.

Afecta C: C13 b (las películas sin positivos tienen $P(i)$ solo por el $+1$), C11 b.

D9. Submuestra o completo

D7, D10

¿Con qué conjunto se desarrolla y con cuál se informa?

a) Submuestra para todo, completo solo al final.

b) ⋆ Submuestra para desarrollar y, antes de cada entrega, repetir las 2–3 configuraciones finales en el completo.

c) Completo desde el principio.

Evaluación: el frío de la submuestra es peor que el real (15 422 de 23 335 películas con < 20 ratings en train); los cortes temporales coinciden. **E2**: parámetro `-muestra`.

Afecta C: C13 b, C8 b.

D10. Semillas y reproducibilidad

D7, H7

¿Cómo hacemos el split reproducible?

a) Una semilla fija (42).

b) ⋆ Asignar cada usuario a su grupo con una función hash estable de `userId` y la semilla; guardar la tabla; repetir los resultados finales con 5 semillas y dar media y desviación.

c) Semilla distinta para cada componente (split, inicialización, orden de lotes).

Evaluación: con 5 semillas, los ratings de test varían $66\,565 \pm 1\,569$ (usuarios) y $315\,203 \pm 223$ (aleatorio). **E2**: la tabla de grupos y todos los umbrales van en el archivo de configuración.

Afecta C: C1 b, C13 b (todo lo que depende de train cambia con la semilla).

Bloque E: features de la torre de película

E1. Embedding de la sinopsis

E8

¿Cómo se convierte el texto en vector?

a) ⋆ MiniLM (`all-MiniLM-L6-v2`, $d = 384$), precalculado.

b) TF-IDF + SVD.

c) Ambos (ablación).

Modelo: la sinopsis ocupa 384 de las entradas de la torre; en `float16` son 17,9 MB para 23 335 películas, en `float32`, 35,8 MB. Mediana de 46 palabras (cabe en la ventana). **Pérdida/evaluación**: ninguna directa; (c) da una comparación gratis. **E2**: caché `.npy`. 127 sinopsis vacías: vector de ceros y bandera.

Afecta C: C13 b (el vector solo describe contenido: no usa ratings).

E2. Créditos

E9, D3

¿Qué parte de director y reparto?

a) Nada.

b) ⋆ Director + 3–5 primeros actores, con frecuencia mínima (p. ej. ≥ 5 películas; el resto a `<OTRO>`) o *hashing*.

c) Todo el reparto y el equipo.

Modelo: 7 827 directores y 22 100 actores (solo 1 114 con ≥ 10 películas; 36,4 % de las apariciones); sin frecuencia mínima, columnas que casi nunca se entrenan. Cobertura: 84,8 % de películas (97,0 % de ratings), pero 9,2 % de las posteriores a 2017. **D3**: la frecuencia mínima y el vocabulario se cuentan con las películas de train.

Afecta C: C13 b (las películas con $P(i) = 0$ dependerán de estas features), C11 b.

E3. Palabras clave

E1

¿Se incluyen?

a) No.

b) ⋆ Añadirlas al texto antes de MiniLM (sin features nuevas).

c) Multi-hot con frecuencia mínima.

Cobertura: 84,7 % de las películas (98,5 % de los ratings); el 91,9 % de las posteriores a 2017 (más que los créditos, porque la API de TMDB sí las devuelve). **Modelo**: (b) no cambia la arquitectura; (c) añade otro vocabulario con la misma pega que E2.

Afecta C: —

E4. Features numéricas

D3, C13

¿Año, duración, popularidad, `vote_average`?

a) ⋆ Año y duración (normalizados con train o por décadas), sin popularidad ni `vote_average`.

b) Todas.

c) (a) más la popularidad solo como sesgo de entrenamiento.

d) Ninguna.

Modelo: año 99,7 % de cobertura; duración 84,7 % (8,9 % en las ≥ 2017). La popularidad empuja al modelo a recomendar lo popular: el 10 % más visto reúne el 78,8 % de los ratings, y con C13 b logQ ya trae esa información a la pérdida. `vote_average` viene de otros usuarios de TMDB y mezcla calidad con popularidad.

Afecta C: C13 b (logQ y popularidad como feature se pisan), C1 b.

E5. Géneros (decisión B3)

E9, inferencia

¿MovieLens o TMDB?

a) Solo MovieLens (19 géneros; 99,0 % de cobertura).

b) Solo TMDB (23 géneros; 98,7 %).

c) ⋆ Unión: TMDB primero y MovieLens como relleno (hay 258 películas con géneros solo en MovieLens), quitando IMAX (formato) y Foreign (no es un género); vocabulario común de 16 mapeado.

d) Solo los 16 géneros comunes.

Modelo: Jaccard medio 0,75; listas idénticas en el 50,2 %. Las películas nuevas y las de Letterboxd solo tendrán géneros de TMDB: entrenar con géneros de MovieLens crea una discrepancia entre entrenamiento y servicio. 38 películas sin géneros en ninguna: vector de ceros y bandera.

Afecta C: C3 c (las anti-preferencias de la torre de usuario usan los géneros de las películas), C10 a.

E6. Identificador de película

D8, F1

¿Embedding de `movieId` o solo contenido?

a) Solo contenido.

b) Contenido + embedding de ID.

c) ⋆ Contenido + ID con *dropout* del ID (y un ID «desconocido» para películas nuevas).

Modelo: tabla de 2 986 880 parámetros frente a 1 491 641 positivos de train; 12 872 películas con < 5 positivos y 4 354 con ninguno. **Evaluación**: se decide con el experimento de D8 d; una película nueva sin ID entrenado solo se sostiene con contenido. **Pérdida**: logQ no cambia, pero el ID permite que el modelo memorice la popularidad.

Afecta C: C13 b, C5 a (sin tope por usuario, el ID aprende sobre todo de los muy activos).

E7. Combinación y normalización

E1, MMR

¿Cómo se juntan las features y se normalizan los vectores?

a) Concatenar y una red densa; sin normalizar $\mathbf{v}_i$.

b) ⋆ Concatenar y red densa; normalizar con L2 $\mathbf{u}$ y $\mathbf{v}_i$ y usar una temperatura T ([8]).

c) Normalizar solo $\mathbf{v}_i$.

Modelo y pérdida: la temperatura es un hiperparámetro de la softmax (D4). **Inferencia**: con vectores normalizados, el producto escalar de HNSW y el coseno del MMR coinciden.

Afecta C: C1 b, C13 b (se resta $\log P(i)$ a la puntuación con T).

E8. Encoder congelado o ajustado

E1

¿Se entrena MiniLM?

a) ⋆ Congelado, con una capa densa entrenable encima.

b) Ajustar la última(s) capa(s).

c) Ajustar todo.

Coste: (a) no entra en el bucle de entrenamiento y se lee con `.npy` desde C++; (b)/(c) obligan a recalcular los 23 335 vectores tras cada cambio.

Afecta C: —

E9. Datos que faltan E2, E4

¿Qué hacemos con las películas sin director, reparto o duración (posteriores a 2017)?

a) ⋆ Token «desconocido» y una bandera `falta_credito`; con *dropout* de los créditos al entrenar (para que el modelo vea películas sin ellos).

b) Descargar los créditos que faltan con la API de TMDB (como se hizo con las sinopsis).

c) Excluir esas películas del entrenamiento.

Modelo: solo el 9,2 % de las películas de 2017 en adelante tienen director; sin tratamiento, la torre aprende «sin director = reciente». **Datos**: (b) cuesta una descarga (3 361 películas) pero cambia la cobertura; se puede hacer después. **Evaluación**: se mira en el experimento de D8 d.

Afecta C: C11 b, C13 b.

Resumen de dependencias

Pregunta	Depende de...	Porque...
D1	F1 (y C1)	el split por usuarios exige torre de usuario basada en historial
D2	C9	el orden en el primer día es el de tecleo
D3	C6, C13, C14	definen qué se calcula con train
D4, D6	C7, C8	las dos definiciones de relevante y los usuarios elegibles
D5	C9	el 59 % de usuarios en un solo día
D8	E6, D1	el frío de películas solo se mide con películas retiradas
E1, E2, E3	D3	vocabularios, frecuencias mínimas, idf: solo con train
E4	C13	logQ y popularidad como feature se pisan
E5	B3	fija la decisión B3
E6	D8, F1	decide si la torre funciona con solo contenido
E9	E2, E4	las películas posteriores a 2017 carecen de créditos

Orden propuesto: (1) D1, F1 y E6 a la vez (qué sabe cada torre y con qué se mide); (2) D3 y E1–E4 (qué se ajusta y con qué datos); (3) D2, D4–D5 y D7 (detalles del split y empates); (4) E5, E7–E9 (géneros, normalización y datos que faltan); (5) D6, D8–D10 al escribir el pipeline de evaluación y el archivo de configuración.

Glosario

Arranque en frío
: Un usuario o una película sin historia en train.

Contexto / *fold-in*
: Parte del historial de un usuario de test que se da al modelo para construir su vector; no se predice.

Elegible
: Usuario de test con ≥ 1 positivo en test y ≥ 1 en su historial visible.

Fit/transform
: Ajustar una transformación con train y aplicar el resultado ya fijado al resto.

Fuga (*leakage*)
: Usar en entrenamiento información que solo existe en test.

Generalización fuerte
: Los usuarios de test no están en el entrenamiento.

Hashing
: Reducir un vocabulario enorme a un tamaño fijo con una función hash.

Leave-last-out
: Último rating de cada usuario a test.

MiniLM
: Red preentrenada que convierte un texto en un vector de 384 números.

Techo de Recall@K
: Máximo Recall@K alcanzable: media de mín$(K, |R_u|)/|R_u|$.

TF-IDF / SVD
: Ponderación de palabras por su rareza y compresión por valores singulares.

Referencias

[1] J. Carbonell, J. Goldstein. *The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries*. SIGIR, 1998.

[2] P. Covington, J. Adams, E. Sargin. *Deep Neural Networks for YouTube Recommendations*. RecSys, 2016.

[3] F. M. Harper, J. A. Konstan. *The MovieLens Datasets: History and Context*. ACM TiiS, 2015.

[4] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, T.-S. Chua. *Neural Collaborative Filtering*. WWW, 2017.

[5] W.-C. Kang, J. McAuley. *Self-Attentive Sequential Recommendation*. ICDM, 2018.

[6] D. Liang, R. G. Krishnan, M. D. Hoffman, T. Jebara. *Variational Autoencoders for Collaborative Filtering*. WWW, 2018.

[7] H. Steck. *Embarrassingly Shallow Autoencoders for Sparse Data*. WWW, 2019.

[8] X. Yi, J. Yang, L. Hong, D. Z. Cheng, L. Heldt, A. Kumthekar, Z. Zhao, L. Wei, E. Chi. *Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations*. RecSys, 2019.